Explainable Fake Job Detection Framework

Repository Structure & Blueprint Guide

This document details the architectural folder structure of the project. The project is structured as a three-tier architecture: a React frontend client, a Node.js/Express orchestration backend gateway, and a Python FastAPI AI computation service, persisted by a MongoDB database. This guide details where specific functions run and where the files are located.

1. Python FastAPI AI Service (ai_service/)

Located inside the ai_service/ directory, this Python environment runs pre-processing cleanups, BERT tokenization, binary ML predictions, heuristic scores, and explains predictions locally utilizing LIME and SHAP models.

File / Subfolder	Function & Execution Details
app.py	Configures the FastAPI HTTP app. Initiates routing hooks like `/analyze/text`, `/analyze/url`, `/analyze/pdf`, `/analyze/image`.
pipeline.py	Contains run_analysis_pipeline() which orchestrates the complete analysis flow (Ingestion -> Preprocessing -> BERT classification -> Trust Score -> Severity Score -> Risk Tiers -> LIME/SHAP).
bert_model/	Houses fine-tuning and inference logic. Loads the fine-tuned BERT-base-uncased classifier weights and evaluates fraud probabilities via predict_fraud_probability().
preprocessing/	Strips HTML symbols, removes emojis, normalizes whitespaces, and runs WordPiece tokenization prior to ML classification.
signals/	Heuristic extraction script detectors.py scans for 10 deterministic flags like fee requests or free email handlers.
trust_engine/	Calculates a 0–100 Trust Score via compute_trust_score() using negative adjustments for heuristic warning triggers.
severity_engine/	Computes a 0–100 Severity Score via compute_severity_score() to assess candidate damage risks (e.g. wire transfer requests).
risk_engine/	Performs 4-tier risk classification via categorize_risk() mapping the composite BERT probability grid against the Trust Score.
xai/	Manages explainability. Loads precomputed SHAP values for global dataset patterns and runs LIME local models live per instance request.
input_processing/	Extracts text from various input channels: uses PyMuPDF for documents, EasyOCR for screenshots, and JSON-LD scrapers for job page URLs.

2. Express Orchestration Backend Gateway (backend/)

Written in JavaScript (Node.js/Express), this layer acts as the public gatekeeper, managing user sessions, enforcing rate limits, validating files, proxying requests to the FastAPI backend, and logging outcomes to MongoDB.

File / Subfolder	Function & Execution Details
app.js / server.js	Initializes database connections, configures CORS, binds middleware routers, and opens the listener port (5000).
routes/	Maps HTTP pathways. Organizes user authorization end routes, history listing routes, and upload endpoints.
controllers/	Houses handlers: analysisController.js receives files, pushes payloads to FastAPI client, and logs outcomes in MongoDB via persistAnalysis(). authController.js handles logins/session JWTs.
models/	Configures MongoDB schemas: `User.js` saves passwords and profiles; `Analysis.js` stores the complete AI evaluation scorecard (scores, text, attributions, recommendations).
middleware/	Binds security handlers: validates JWT sessions, applies rate limits, and uses Multer to validate and buffer PDF or image uploads.
utils/aiServiceClient.js	An Axios HTTP wrapper that handles communications and transfers raw parameters between Node.js and the FastAPI server.

3. React Frontend Dashboard Client (frontend/)

Written in TypeScript / React and styled with Tailwind CSS, this layer renders the user interface and handles multi-input upload submission interfaces.

File / Subfolder	Function & Execution Details
App.tsx	Declares the frontend routing paths, rendering landing panels, analysis components, scan histories, and profiles.
src/pages/	Main screen components. AnalysisPage.tsx renders text/URL inputs, file selectors, and scorecards. HistoryPage.tsx lists past scans.
src/components/	Reusable widgets. LimeHighlight.tsx highlights words in red or green based on local weights. ShapChart.tsx charts global feature weights.
src/api/	Houses API utilities to handle requests and capture responses from the Express gateway endpoint.